

Generative Feature Re-Engineering in Microservice Architectures

Johan Martinson^{1,2}[0000–0002–4097–4374]

¹ Ruhr University Bochum, Bochum, Germany

² Centiro Solutions AB, Gothenburg, Sweden
`johan.martinson@rub.de`

Abstract. Managing variant-rich systems from a single, evolving microservice codebase demands that developers know where features live, how their implementations are distributed across services, and how to make a located feature optional at runtime. Each layer compounds the previous: feature location fades as knowledge leaves the organization, centralized feature models do not scale across independently-owned repositories, and the re-engineering step that turns a located feature into a runtime variation point has no automated support. We present an ongoing PhD research program that develops and evaluates GRETA (Generative Re-Engineering via Traceability Annotations), an LLM-driven workflow in which AI agents guided by embedded feature annotations and a distributed feature model automatically introduce runtime variation points into existing microservice code. The approach is validated through a longitudinal case study at an industrial logistics platform running more than one hundred microservices. Preliminary results show that annotation recording costs 6.27% of task time with negligible workload increase, that agent-maintained annotations are achievable at zero marginal cost, that commercial LLMs succeed at featurization in 46.7% of cases without fine-tuning, and that decentralized per-repository feature modeling is theoretically sound. These results establish the annotation and model foundations on which the GRETA re-engineering agent is being built.

Keywords: feature-oriented software engineering · embedded feature annotations · variability management · software product lines · large language models

1 Introduction and Motivation

Whether to satisfy a new customer or to fix a defect, developers constantly add, change, and remove *features*—abstractions of functionality that are meaningful to an end-user [1, 11]. To do so, they must know where in the code each affected feature lives. Consider a logistics platform that serves hundreds of customers, each enabling a different combination of capabilities such as track-and-trace visibility, proof-of-delivery capture, carrier integrations, and compliance-driven audit logging. This is the organization our longitudinal case study draws on

(Sect. 4). Onboarding a customer means deciding which features that customer needs, locating the services that implement them, and switching them on. As requirements evolve, features are continuously added, changed, and removed [2], and every change depends on knowing where the affected feature lives.

Serving many customers from one evolving codebase is a variability problem, and managing it systematically is the central concern of software product-line engineering [4, 27]. The established response is to make features explicit, recording them and their traces to code as first-class development artifacts rather than leaving them implicit in the structure of the codebase [24]. Organizations identify their features, organize them in a feature model that captures their relationships and constraints [11, 1], record their locations in the source code [33, 19, 10], and derive individual variants by selecting features in a configurator. Organizations that have adopted this discipline, and where they sustain it, it delivers reuse across products and faster time-to-market [4].

Unfortunately, adoption remains elusive in fast-moving organizations with large, evolving codebases. The discipline competes directly with feature delivery, since annotating code and maintaining models pays off only later, whereas shipping features pays off now. As a result, developers rarely record where features live as they implement them, and knowledge about feature locations fades over time [2]. Recovering that knowledge afterwards is laborious and error-prone, especially once the engineers responsible for a feature have left the organization [2]. Both automated and manual feature-location techniques struggle to recover locations retroactively, with automated approaches producing too many false positives to scale to large systems [31, 5] and manual recovery remaining effort-intensive [13].

These difficulties compound in microservice architectures. Feature scattering, where a single feature is implemented across many fragments of code, is a long-studied obstacle to comprehension and maintenance [26]. In large microservice systems the same phenomenon appears at architectural scale, with one feature spread across many of the hundreds of repositories that make up the system [25]. Feature modeling, however, has traditionally remained centralized, and a single model cannot span such a system without a central authority to coordinate cross-repository constraints, a challenge existing techniques do not address [22].

Even once feature locations are recorded and distributed models are in place, a final obstacle remains. Making an existing feature optional, or producing an optional variant of it, still demands manual re-engineering, since developers must trace and adapt every code path that implements the feature before it can be switched on or off at runtime. This last step, turning located features into configurable ones, is where the cost of variability concentrates.

Our long-term goal is to raise the level of abstraction at which variant-rich systems are managed, letting organizations operate at the feature level rather than the code level. Over the course of this PhD we have built tooling for the location and modeling levels of this vision. HANS [19] provides IDE support for recording feature locations as embedded annotations [33], and HANS-VIZ [18, 7] makes annotation coverage and feature evolution visible during development,

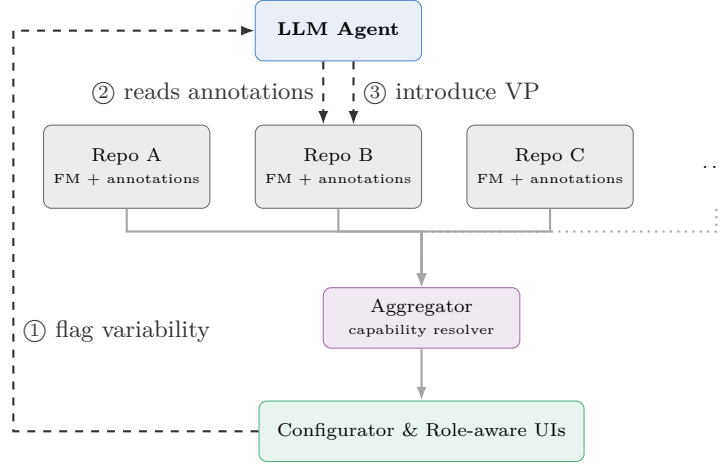


Fig. 1. Target system architecture and re-engineering workflow. Microservice repositories carry per-repo feature models and embedded annotations (RQ1). The aggregator composes them into a distributed feature model (RQ2), which drives the configurator and role-aware UIs. When the configurator flags a variability need (①), an LLM agent reads the relevant annotations (②) and introduces a runtime variation point into the microservice code (③) — this re-engineering loop is the focus of RQ3.

giving developers an immediate payoff for recording locations. FM-PRO [20] offers systematic guidance for constructing feature models, and recent work shows that large language models can identify variability opportunities and suggest where annotations should be introduced in existing code [16], lowering the upfront cost of adoption. To the best of our knowledge, none of these address the re-engineering obstacle, and no existing approach uses feature annotations and distributed models to automatically introduce runtime variation points into microservice code. We call this workflow GRETA (*Generative Re-Engineering via Traceability Annotations*).

Software development is shifting toward agentic coding, where developers steer LLM agents from the prompt window and review the resulting change-sets [12, 32, 6]. Because an agent is already in the loop, maintaining feature annotations and introducing variation points can become part of its instructions rather than a separate developer task [16]. A longitudinal case study at an organization running more than one hundred microservices provides the scale and operational stakes to test whether these developments together lower the adoption barrier.

Research objective. *Our research objective is to develop and evaluate an LLM-driven re-engineering workflow in which AI agents, guided by embedded feature annotations and distributed feature models, automatically introduce runtime variation points into existing microservice code.*

2 Research Questions

The organizing goal of this research is automated re-engineering via an LLM agent that turns a located feature into a runtime variation point (RQ3). That goal is only achievable when an agent can locate features in code (RQ1) and reason about their cross-service dependencies (RQ2). The three research questions are therefore ordered by dependency, not equal weight.

RQ1 — Annotation cost. *How can the cost of embedded feature annotation be kept low enough that an industrial microservice organization maintains annotation coverage without a dedicated annotation maintainer?*

Annotation-based approaches have been piloted in industry but have not achieved broad adoption [2]. The barrier is economic. Recording and maintaining annotations takes time that competes with feature delivery, yet developers rarely record where features live as they implement them, and knowledge about feature locations fades over time [2]. Existing IDE tooling records annotations during development and makes coverage visible in the IDE [19, 18, 7]. A controlled experiment with 15 professional developers found that annotation recording costs 6.27% of task time with negligible workload increase, yet only 53% of participants stated they would adopt annotations in their regular workflow [17]. AGENT-HANS addresses both sides of this value-adoption gap by identifying where annotations should be introduced and maintaining them at zero marginal cost after every code-generation step [16]. What remains open is whether these together lower costs to a level the organization sustains without a dedicated annotation maintainer, and whether the annotation coverage they produce is sufficient for the re-engineering agent in RQ3.

RQ2 — Decentralized feature models. *How can feature models for more than one hundred independently-owned microservice repositories be represented and aggregated without a single monolithic model?*

Centralized feature models do not scale to large multi-team, multi-repository systems, as coordination overhead, size, and ownership conflicts become prohibitive, and existing multi-level approaches still assume a coordinating model [22, 29, 14, 28]. We have proposed a decentralized approach [22, 15] in which each repository carries a small feature model scoped to its own assets, and cross-repository dependencies are expressed via explicit capability constraints. RQ2 asks whether this approach is implementable at the scale of more than one hundred independently-owned repositories, what the scaling characteristics are, and what process friction arises in practice. The answers bound what aggregate model fidelity the re-engineering agent in RQ3 can rely on.

RQ3 — Re-engineering workflow. *How can LLM agents, guided by embedded feature annotations and a distributed feature model, correctly introduce runtime variation points into existing microservice code?*

Making a feature optional concentrates variability cost at the code level. Every code path that implements the feature must be located, traced across service boundaries, and adapted before the feature can be switched at runtime. This last

step has no automated support, and the effort scales with the number of services affected (Figure 1). At the same time, software development is shifting toward agentic coding, where developers steer LLM agents through the prompt window and review the resulting changesets [12, 32, 6]. Because an agent is already in the loop, introducing variation points can become part of its instructions rather than a separate developer task. The annotations and the distributed feature model are intended to make this approach feasible, giving the agent the context it needs to locate the right code fragments and understand cross-service constraints. We have shown on four small open-source systems that LLMs can produce promising variability-implementation suggestions without fine-tuning [21], though success varied by scenario and model. RQ3 asks how well LLM agents can introduce a variation point with the correct variability mechanism [3, 30]. The central concern is the correctness of the variation point’s implementation and the semantics of the resulting solution.

3 Research Methodology and Approach

Our methodology follows design science [8, 9], an iterative approach of problem awareness, artifact design, and validation. Each research question produces one or more artifacts — tools, processes, or empirical studies — that address a concrete problem and are evaluated in practice.

Design science cycles. The research proceeds in three cycles, one per RQ. Each cycle constructs an artifact based on requirements elicited from the industrial case study, evaluates the artifact empirically, and feeds lessons into the next cycle. Artifacts from earlier cycles feed later ones, though the cycles are not strictly sequential. Annotation tooling (RQ1) underpins the aggregator (RQ2), while RQ3 consumes embedded annotations and per-repository feature models directly and need not wait for the full aggregator.

Industrial longitudinal case study. The validation setting is a logistics-software company operating more than one hundred microservices in C#/.NET and TypeScript/Angular, serving hundreds of customers with distinct feature combinations. This setting provides three things a controlled study alone cannot: *(i)* scale (100+ repositories), *(ii)* operational stakes (real customer onboarding), and *(iii)* longitudinal depth (repeated measurement as the system and team evolve). The company has participated in studies during the course of the research, and familiarity with the tooling reduces onboarding friction and allows us to study adoption over time rather than in a one-shot experiment.

Feature modeling principles. We draw on established feature modeling principles [23], the FM-PRO process [20] for systematic feature-model construction and maintenance, and the cost-benefit model of [2] as a baseline for measuring adoption improvement.

4 Preliminary Results

We report preliminary results from six completed or submitted studies.

Annotation Overhead: Controlled Experiment. We conducted a controlled experiment with 15 professional C# developers to measure the time and workload cost of recording embedded feature annotations with HANS during development [17]. Participants completed a real development task on an industrial freight-calculation system, half with annotation recording and half without, in a counterbalanced design.

Annotation recording consumed $6.27\% \pm 3.26\%$ of total task time (median 5.29%; absolute mean 1.18 ± 0.51 minutes per task), with a small time effect of Cohen’s $d = 0.265$. NASA-TLX workload scores showed a negligible difference (Cohen’s $d = 0.06$) and annotations did not meaningfully increase perceived workload. **93%** of participants agreed that annotations are valuable for future maintenance, yet only **53%** stated they would adopt them in their regular workflow. This value-adoption gap motivates the cost-reduction work of RQ1, alongside the companion visualization HANS-VIZ [18, 7], which makes annotation coverage and feature evolution visible during development as an immediate payoff.

Agent-Maintained Annotations. AGENT-HANS [16] is an agentic coding workflow and MCP server that instructs an LLM coding agent to update the feature model and embedded annotations after every code-generation step. A demonstration on a C#/.NET web API built entirely via LLM agent (27 prompts across 10 evolution steps) produced 25 features covering four evolution types, *(i)* additions, *(ii)* a merge, *(iii)* a split, and *(iv)* a removal, with zero manual annotation effort between cycles. This demonstration suggests that zero-marginal-cost annotation maintenance is achievable.

LLM-Assisted Featurization. We evaluated popular LLMs on their ability to *featurize* (make a feature explicit and optional) existing code using zero- and few-shot prompting [21]. Experiments covered four open-source Java systems across six featurization scenarios, each requiring a feature to be made optional using one of two variability mechanisms: parameters or design patterns. Fig. 2 shows which prompting strategy was most effective per scenario and model. Without fine-tuning, commercial LLMs made the feature optional in 46.7% of cases against 11.7% for the open-source models, with success varying sharply by scenario; zero-shot prompting alone was not reliable enough for an assisted tool. These results directly inform the LLM-assisted featurization study in RQ3.

Decentralized Feature Modeling. We proposed a decentralized approach to feature modeling in which multiple per-repository feature models coexist, each scoped and owned independently, composed via capability-based cross-model constraints [22]. A semi-systematic review confirmed that no existing modular or multi-level feature-modeling technique fully addresses independent configuration and incremental analysis together. This vision paper provides the theoretical foundation and design-space requirements for the RQ2 aggregator.

Featurization Criteria: FM-PRO. FM-PRO [20] is a feature-modeling process evaluated with 6 industrial practitioners and 27 experts in feature modeling. Its evaluation identified essential criteria for successful featurization, including feature scoping, cross-asset consistency, and evolution traceability. These criteria



Fig. 2. Most effective prompting strategy per scenario and LLM model (zero- and few-shot); cell number identifies the winning strategy, color encodes its success score, from [21]. Evaluated across four open-source Java systems and six featurization scenarios.

inform the quality requirements for both the annotation guidelines (RQ1) and the aggregate model semantics (RQ2).

5 Work Plan

Table 1 situates the work already completed (Sec. 4) within the schedule for the next 12 months. Four milestones run in parallel over the coming year, each targeting a peer-reviewed publication.

AgentHANS improvements and usability study (M1–M6). We extend AGENT-HANS beyond the single-system demonstration [16] and evaluate it with professional developers using SUS and NASA-TLX. The study measures the accuracy of agent-produced annotations and whether developers accept maintenance delegated to the agent, targeting the 53% value-adoption gap left open by the overhead experiment [17].

Aggregator implementation and scaling study (M2–M8). We implement the aggregator specified by the decentralized vision paper [22] and deploy it on a proof-of-concept subset of three to six case-study microservices, kept small enough to hand-curate a ground truth while still exercising cross-service aggregation. Evaluation compares the aggregated model against that ground truth for correctness, aggregation latency, and capability-resolution accuracy, establishing whether decentralized feature models hold at this scale.

Longitudinal annotation-adoption case study (M4–M10). Drawing on the company’s prior HANS familiarity, we track annotation coverage across real feature delivery over six months. The study tests whether AGENT-HANS lowers cost enough that coverage is sustained without a dedicated annotation maintainer, the central claim of RQ1. We also build a feature-model-driven configu-

Table 1. Research work plan. Each milestone produces a planned publication (P).

Period	RQ	Milestone	Target
Done	RQ1	Annotation overhead experiment [17]	VARIABILITY’26
Done	RQ1	AGENT-HANS tool demonstration [16]	ASE’26
Done	RQ1	FM-PRO [20]	TSE’25
Done	RQ1	Feature-evolution visualization [7]	SPLC’25
Done	RQ2	Decentralized FM vision [22]	VARIABILITY’26
Done	RQ3	LLM featurization experiment [21]	AiSoLa’24
M1–6	RQ1	AgentHAnS improvements and usability study	
M2–8	RQ2	Aggregator implementation and scaling study	
M4–10	RQ1	Longitudinal case-study annotation adoption	
M1–12	RQ3	Runtime variation-point study through LLM re-engineering	

rator that lets developers toggle features at runtime without code changes, and measure whether this payoff increases annotation adoption.

Runtime variation-point study (M1–M12). We construct a taxonomy of variation-point patterns for the C#/.NET and Angular/TypeScript stack, then build the LLM agent workflow that re-engineers a located feature into a variation point, guided by embedded annotations and the distributed feature model as context. We check correctness against the services’ existing test suites together with characterization tests that capture each component’s behavior before re-engineering, so the agent is never graded by tests it generated. Evaluation measures the fraction of variation points the agent introduces correctly and which patterns resist automation. Effort is measured against a manual baseline in which the researcher guides company developers through the same re-engineering using FM-PRO [20]. This study is the core RQ3 contribution and runs the full year.

Thesis. The thesis synthesizes the three questions into a longitudinal industrial case study, documenting the programme from annotation tooling through automated re-engineering.

6 Conclusion

We presented a doctoral research program organized around three research questions, covering the cost of keeping annotations current, the representation of feature models across more than one hundred independently-owned repositories, and the automated re-engineering itself. Six completed studies, detailed in Section 4, establish the annotation and model foundations on which the GRETA re-engineering agent is being built. Over the coming year we implement the milestones outlined in Section 5, with the aim of lifting the level of abstraction at which variant-rich systems are managed from the code level to the feature level, letting organizations operate on features rather than the code that implements them. Our tools and methods are open source. The industrial case-study data remains confidential to the company.

References

1. Berger, T., Lettner, D., Rubin, J., Grünbacher, P., Silva, A., Becker, M., Chechik, M., Czarnecki, K.: What is a feature? a qualitative study of features in industrial software product lines. In: SPLC. pp. 16–25. SPLC '15 (2015). <https://doi.org/10.1145/2791060.2791108>
2. Berger, T., Mahmood, W., Abu Zahra, R., Vassilevski, I., Burger, A., Ji, W., Antkiewicz, M., Czarnecki, K.: Cost and benefit of tracing features with embedded annotations. *ACM Transactions on Software Engineering and Methodology* **35**(4) (2026). <https://doi.org/10.1145/3746060>
3. Berger, T., Pfeiffer, R.H., Tartler, R., Dienst, S., Czarnecki, K., Wąsowski, A., She, S.: Variability mechanisms in software ecosystems. *Information and Software Technology* **56**(11), 1520–1535 (2014)
4. Clements, P., Northrop, L.: *Software product lines*. Addison-Wesley Boston (2002)
5. Dit, B., Revelle, M., Gethers, M., Poshyvanyk, D.: Feature location in source code: a taxonomy and survey. *Journal of Software: Evolution and Process* **25**(1), 53–95 (2013). <https://doi.org/https://doi.org/10.1002/smr.567>, <https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.567>
6. Hassan, A.E., Li, H., Lin, D., Adams, B., Chen, T.H., Kashiwa, Y., Qiu, D.: *Agentic Software Engineering: Foundational Pillars and a Research Roadmap* (2025)
7. Hermann, K., Martinson, J., Berger, T.: Visualizing feature-oriented software evolution. In: SPLC. pp. 136–141 (2025). <https://doi.org/10.1145/3744915.3748471>
8. Hevner, A., Chatterjee, S.: *Design Science Research in Information Systems*. Springer US (2010)
9. Hevner, A.R., Storey, V.C.: Externalities of design science research: Preparation for project success. In: *The Next Wave of Sociotechnical Design: 16th International Conference on Design Science Research in Information Systems and Technology, DESRIST 2021, Kristiansand, Norway, August 4–6, 2021, Proceedings 16*. pp. 118–130. Springer (2021)
10. Ji, W., Berger, T., Antkiewicz, M., Czarnecki, K.: Maintaining feature traceability with embedded annotations. In: SPLC (2015)
11. Kang, K.C., Cohen, S.G., Hess, J.A., Novak, W.E., Peterson, A.S.: Feature-oriented domain analysis (FODA) feasibility study. Tech. rep., CMU SEI (1990)
12. Karpathy, A.: <https://x.com/karpathy/status/1886192184808149383> (2025)
13. Krüger, J., Berger, T., Leich, T.: *Features and How to Find Them: A Survey on Manual Feature Location*, pp. 153–172. LLC/CRC Press, Boca Raton, FL, USA (01 2019). <https://doi.org/10.1201/9780429022067-7>
14. Lettner, D., Grünbacher, P.: Using feature feeds to improve developer awareness in software ecosystem evolution. In: *Proceedings of the 9th International Workshop on Variability Modelling of Software-intensive Systems*. pp. 11–15. VaMoS '15, ACM, Hildesheim, Germany (2015). <https://doi.org/10.1145/2701319.2701331>
15. Mahmood, W., Strueber, D., Berger, T., Lämmel, R., Mukelabai, M.: Seamless variability management with the virtual platform. In: ICSE (2021)
16. Martinson, J., Bengtsson, J., Hultin, M., Hermann, K., Berger, T.: Agent-HANs: Maintaining feature and code traceability in agentic development. In: ASE. ACM (2026), to appear
17. Martinson, J., Berger, T.: Overhead of recording feature locations with embedded annotations. In: *Variability. LNCS*, Springer (2026), to appear
18. Martinson, J., Hermann, K., Houbbi, R., Stechow, D., Berger, T.: Lightweight visualization of software features with HANs-viz. In: SPLC. pp. 31–34 (2025). <https://doi.org/10.1145/3748269.3748487>

19. Martinson, J., Jansson, H., Mukelabai, M., Berger, T., Bergel, A., Ho-Quang, T.: Hans: Ide-based editing support for embedded feature annotations. In: Proceedings of the 25th ACM International Systems and Software Product Line Conference-Volume B. pp. 28–31 (2021)
20. Martinson, J., Mahmood, W., Gyimah, J., Berger, T.: Fm-pro: A feature modeling process. *IEEE Transactions on Software Engineering* **51**(1), 262–282 (2025). <https://doi.org/10.1109/TSE.2024.3513635>
21. Martinson, J., Noller, Y., Berger, T.: On using large language models to ‘featurize’ software. In: AISoLA. LNCS, Springer (2024)
22. Mukelabai, M., Martinson, J., Mahmood, W., Berger, T., Capilla, R.: Towards decentralized feature models. In: Variability. LNCS, Springer (2026)
23. Nesic, D., Krueger, J., Stanculescu, S., Berger, T.: Principles of feature modeling. In: FSE (2019)
24. Passos, L., Czarnecki, K., Apel, S., Wąsowski, A., Kästner, C., Guo, J.: Feature-oriented software evolution. In: Proceedings of the 7th International Workshop on Variability Modelling of Software-Intensive Systems. VaMoS ’13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2430502.2430526>, <https://doi.org/10.1145/2430502.2430526>
25. Passos, L., Padilla, J., Berger, T., Apel, S., Czarnecki, K., Valente, M.T.: Feature scattering in the large: A longitudinal study of linux kernel device drivers. In: 14th International Conference on Modularity (MODULARITY) (2015)
26. Passos, L., Queiroz, R., Mukelabai, M., Berger, T., Apel, S., Czarnecki, K., Padilla, J.A.: A study of feature scattering in the linux kernel. *IEEE Transactions on Software Engineering* **47**(1), 146–164 (2021). <https://doi.org/10.1109/TSE.2018.2884911>
27. Pohl, K., Böckle, G., Van Der Linden, F.: Software product line engineering: Foundations, Principles, and Techniques, vol. 10. Springer (2005)
28. Prähofer, H., Rabiser, D., Angerer, F., Grünbacher, P., Feichtinger, P.: Feature-oriented development in industrial automation software ecosystems: Development scenarios and tool support. In: Proceedings of the IEEE International Conference on Industrial Informatics (INDIN). pp. 1218–1220. IEEE (2016). <https://doi.org/10.1109/INDIN.2016.7819353>
29. Rabiser, D., Prähofer, H., Grünbacher, P., Petruzelka, M., Eder, K., Angerer, F., Kromoser, M., Grimmer, A.: Multi-purpose, multi-level feature modeling of large-scale industrial software systems. *Software & Systems Modeling* **17**, 913–938 (2018)
30. Rahman, M.T., Querel, L.P., Rigby, P.C., Adams, B.: Feature toggles: practitioner practices and a case study. In: Proceedings of the 13th international conference on mining software repositories. pp. 201–211 (2016)
31. Rubin, J., Chechik, M.: A Survey of Feature Location Techniques, pp. 29–58. Springer, Berlin, Heidelberg, Germany (2013). https://doi.org/10.1007/978-3-642-36654-3_2, https://doi.org/10.1007/978-3-642-36654-3_2
32. Sapkota, R., Roumeliotis, K.I., Karkee, M.: Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai (2025)
33. Schwarz, T., Mahmood, W., Berger, T.: A common notation and tool support for embedded feature annotations. In: SPLC. pp. 5–8. SPLC ’20, Association for Computing Machinery (2020). <https://doi.org/10.1145/3382026.3431253>, <https://doi.org/10.1145/3382026.3431253>